

Experiment 04:

7. Laboratory Exercise

Code:

```
# Extended Euclidean Algorithm
def extended_euclidean_algorithm(first_number, second_number):
    if second_number == 0:
        return first_number, 1, 0

    greatest_common_divisor, first_coefficient, second_coefficient = (
        extended_euclidean_algorithm(
            second_number,
            first_number % second_number
        )
    )
    new_first_coefficient = second_coefficient
    new_second_coefficient = (
        first_coefficient
        - (first_number // second_number) * second_coefficient
    )
    return (
        greatest_common_divisor,
        new_first_coefficient,
        new_second_coefficient
    )

# ----- KEY GENERATION -----

prime_number_p = int(input("Enter prime number p: "))
prime_number_q = int(input("Enter prime number q: "))
public_exponent = int(input("Enter public key e: "))

# Calculate n = p x q
modulus_n = prime_number_p * prime_number_q

# Calculate phi(n) = (p - 1)(q - 1)
phi_n = (prime_number_p - 1) * (prime_number_q - 1)

# Check whether e and phi(n) are coprime
greatest_common_divisor, _, _ = extended_euclidean_algorithm(
    public_exponent,
    phi_n
)

if greatest_common_divisor != 1:
    print("Invalid value of e.")
    print("e and phi(n) must be coprime.")
    exit()

# Calculate d using Extended Euclidean Algorithm
_, private_exponent, _ = extended_euclidean_algorithm(
    public_exponent,
    phi_n
)

# Make d positive
private_exponent = private_exponent % phi_n

# Display keys
print("\n----- KEY GENERATION -----")
print("n =", modulus_n)
```

```

print("phi(n) =", phi_n)
print("Public Key =", (public_exponent, modulus_n))
print("Private Key =", (private_exponent, modulus_n))

# ----- ENCRYPTION -----|

plaintext_message = int(input("\nEnter message P: "))

if plaintext_message >= modulus_n:
    print("Message must be less than n.")
    exit()

# RSA Encryption Formula:
# C = P^e mod n
ciphertext_message = (
    plaintext_message ** public_exponent
) % modulus_n

print("\n----- ENCRYPTION -----")
print("Plaintext =", plaintext_message)
print("Ciphertext =", ciphertext_message)

# ----- DECRYPTION -----

# RSA Decryption Formula:
# P = C^d mod n
decrypted_message = (
    ciphertext_message ** private_exponent
) % modulus_n

print("\n----- DECRYPTION -----")
print("Ciphertext =", ciphertext_message)
print("Decrypted message =", decrypted_message)

```

Exercise 1:

If party A and party B wish to use RSA to communicate securely. 'A' chooses public key (e, n) as (13, 77) and 'B' chooses public key (e, n) as (3, 33). Find

- Value of prime numbers p, q, $\phi(n)$, public key for party A.
- Calculate A's Private key.
- Value of prime numbers p, q, $\phi(n)$, public key for party B.
- Calculate B's Private Key.

Output:

```

Enter prime number p: 7
Enter prime number q: 11
Enter public key e: 13

----- KEY GENERATION -----
n = 77
phi(n) = 60
Public Key = (13, 77)
Private Key = (37, 77)

```

Output:

```

Enter prime number p: 3
Enter prime number q: 11
Enter public key e: 3

----- KEY GENERATION -----
n = 33
phi(n) = 20
Public Key = (3, 33)
Private Key = (7, 33)

```

e. What will be the cipher-text sent by A to B, if A wishes to send Message, $M=5$ to B.

g. What will be the cipher-text sent by B to A, if B wishes to send $M=3$ to A.

f. Show how B will decrypt the message.

h. Show how A will decrypt the message.

Output:

```
Enter message P: 5

----- ENCRYPTION -----
Plaintext = 5
Ciphertext = 26

----- DECRYPTION -----
Ciphertext = 26
Decrypted message = 5
```

Output:

```
Enter message P: 3

----- ENCRYPTION -----
Plaintext = 3
Ciphertext = 27

----- DECRYPTION -----
Ciphertext = 27
Decrypted message = 3
```

Post Experimental Exercise:**Exercise 2:**

If party A and party B wish to use RSA to communicate securely. 'A' chooses public key (e, n) as $(7, 247)$ and 'B' chooses public key (e, n) as $(5, 221)$. Find

a. Value of prime numbers $p, q, \phi(n)$, public key for party A.

c. Value of prime numbers $p, q, \phi(n)$, public key for party B.

b. Calculate A's Private key.

d. Calculate B's Private Key.

Output:

```
Enter prime number p: 13
Enter prime number q: 19
Enter public key e: 7

----- KEY GENERATION -----
n = 247
phi(n) = 216
Public Key = (7, 247)
Private Key = (31, 247)
```

Output:

```
Enter prime number p: 13
Enter prime number q: 17
Enter public key e: 5

----- KEY GENERATION -----
n = 221
phi(n) = 192
Public Key = (5, 221)
Private Key = (77, 221)
```

e. What will be the cipher-text sent by A to B, if A wishes to send Message, $M=5$ to B.

g. What will be the cipher-text sent by B to A, if B wishes to send $M=3$ to A.

f. Show how B will decrypt the message.

h. Show how A will decrypt the message.

Output:

```
Enter message P: 5

----- ENCRYPTION -----
Plaintext = 5
Ciphertext = 73

----- DECRYPTION -----
Ciphertext = 73
Decrypted message = 5
```

Output:

```
Enter message P: 3

----- ENCRYPTION -----
Plaintext = 3
Ciphertext = 22

----- DECRYPTION -----
Ciphertext = 22
Decrypted message = 3
```